A SIMPLE REVIEW OF AUDIO STEGANOGRAPHY IN JAVA

A Mini-Project Report submitted to
Jawaharlal Nehru Technological University
in partial fulfillment of the requirements for the award of Degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

Submitted by

GOSHIKA VAISHNAVI **22WJ8A05NE**

Under the Guidance of

Mrs. B. SUPRAJA

Assistant Professor

Department of Computer Science & Engineering
School of Engineering & Technology
Guru Nanak Institutions Technical Campus
Ibrahimpatnam, Hyderabad, R.R. District – 501506
November, 2025

CERTIFICATE

This is to certify that the Mini-project entitled **"A SIMPLE REVIEW OF AUDIO STEGANOGRAPHY"** is being submitted by **MS. GOSHIKA VAISHNAVI,** bearing Roll No. **22WJ8A05NE** in partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by him under my guidance and supervision.

The results embodied in this Mini-project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Internal Guide
Mrs. B. SUPRAJA
Assistant Professor

Project Coordinator
Dr. N. Naveen Reddy
Assistant Professor

Head of the Department
Dr. M V Narayana

External Examiner

RAM Innovative Infotech

M : +91 9581 012 012
E : raminnovativeinfotech@gmail.com
Flat No.#309, Amrutha Ville,
Opp: Yashoda Hospital, Somajiguda,
Hyderabad-82, Telangana, India
Www.raminnovativeinfotech.webs.com

PROJECT COMPLETION CERTIFICATE

This is to certify that the following students of 3rd year B. Tech, Department of **Computer Science and Engineering** – Guru Nanak Institutions Technical Campus have completed her training and project at GNITC successfully.

STUDENT NAME	ROLL NO
Goshika Vaishnavi	**22WJ8A05NE**

The training was conducted on **Java** Technology for the completion of the Mini-project titled **"A SIMPLE REVIEW OF AUDIO STEGANOGRAPHY IN JAVA"** in **October 2025.** The Mini-project has been completed in all aspects.

Signature

Declaration of Student

I, GOSHIKA VAISHNAVI (22WJ8A05NE), hereby declare that he Mini-project titled "A SIMPLE REVIEW OF AUDIO STEGANOGRAPHY IN JAVA" has been carried out by me as part of the requirements for the award of the Degree of Bachelor of Technology in the Department of Computer Science & Engineering at Guru Nanak Institutions Technical Campus.

I confirm the following:

1. The Mini-project was undertaken by me under the supervision of our guide, Mrs. B. Supraja, from the selection of the topic to the completion of the final report.
2. I have ensured that the results presented in the report are accurate and based on my original work.
3. To the best of my knowledge, the content of this report is free from plagiarism and adheres to ethical standards.
4. The entire mini-project work has been conceptualized, implemented and documented by me.
5. The Mini-project report has been prepared with diligence, ensuring clarity, accuracy, and adherence to academic standards.

I further declare that this report has not been submitted, in part or full, to any other institution or university for the award of any degree or diploma.

GOSHIKA VAISHNAVI

22WJ8A05NE Signature

Date: 01/11/2025

Place: Ibrahimpatnam

Declaration of Guide

I, Mrs. B. Supraja, hereby declare that I have guided the Mini-project titled "A SIMPLE REVIEW OF AUDIO STEGANOGRAPHY" undertaken by GOSHIKA VAISHNAVI (22WJ8A05NE) This Mini-project was carried out towards the fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science & Engineering at Guru Nanak Institutions Technical Campus.

As the guide, I confirm the following:

1. I have overseen the entire Mini-project process, from the selection of the Mini-project title to the submission of the final report.
2. I have reviewed and certified the accuracy and relevance of the results presented in the report.
3. The contributions of the student have been appropriately recognized and assessed.
4. The Mini-project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this Mini-project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of Guide**: Mrs. B. Supraja** Signature of the Guide

Date: 01/11/2025

Place: Ibrahimpatnam

Name of HOD: **Dr. M V Narayana** Signature of the HOD

Department: Computer Science & Engineering

Date: 01/11/2025

Place: Ibrahimpatnam

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to our internal guide, **Mrs. B. Supraja**, Assistant Professor, Department of Computer Science & Engineering, for his valuable guidance, encouragement, and continuous support throughout the duration of this Mini-Project.

I are also thankful to **Dr. M V Narayana**, Professor and HoD, Computer Science & Engineering Department, for his expert supervision and helpful suggestions, which contributed significantly to the successful completion of this Mini-Project.

I would also like to thank the faculty members of the **Computer Science & Engineering Department** and the Lab Technicians for their assistance and cooperation during the practical work of our Mini-Project.

I am grateful to my friends and well-wishers for their encouragement, collaboration, and useful feedback throughout the Mini-Project journey.

Lastly, I sincerely thank my parents for their constant support, patience, and motivation, which helped me complete this Mini-Project successfully.

GOSHIKA VAISHNAVI 22WJ8A05NE

ABSTRACT

Steganography is very important because it conceals the existence of communication itself. Nowadays, people use a variety of multimedia files as steganography carriers. Recently, audio as a type of steganography carrier has attracted more and more attention because of its unique characteristics. The growing popularity of VOIP has also pushed audio into more applications. Audio information hiding technology has its strengths and weaknesses. This paper begins with a brief introduction of basic steganography methods. Then, the related evaluation indexes of audio steganography are introduced. Next, the paper summarizes the performance of main audio steganography methods. Finally, the paper concludes with recommendations and puts forward suggestions for the coming research focus.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
CERTIFICATE	i
PROJECT COMPLETION CERTIFICATE	ii
DECLARATION OF STUDENT	iii
DECLARATION OF GUIDE	iv
ACKNOWLEDGEMENT	v
ABSTRACT	vi
LIST OF FIGURES	x
LIST OF SYMBOLS	xi- xiii
LIST OF ABBREVIATIONS	xiv
1. **INTRODUCTION**	1
1.1 General	1
1.2 Objective	1
1.3 Existing System	2
1.3.1 Existing System Disadvantages	2
1.3.2 Literature Survey	2-6
1.4 Proposed System	7
1.4.1 Proposed System Advantages	7
2. **PROJECT DESCRIPTION**	**8**
2.1 **General**	8
2.2 **Methodologies**	8
2.2.1 Modules Name	8
2.2.2 Module explanation	9
2.2.3 Module Diagram	10-12
2.2.4 Given Input Expected Output	12
2.3 **Techniques Used Or Algorithm Used**	13
3. **REQUIRMENTS ENGINEERING**	14
3.1 **General**	14
3.2 **Hardware Requirements**	14
3.3 **Software Requirements**	15

3.4 **Functional Requirements** 15
3.5 **Non- Functional Requirements** 16

4. **DESIGN ENGINEERING** **17**
4.1 **General** 17
4.2 **Use Case Diagram** 18
4.3 **Class Diagram** **19**
4.4 **Object Diagram** **20**
4.5 **State Chart Diagram** **21**
4.6 **Sequence Diagram** **22**
4.7 **Collaboration Diagram** **23**
4.8 **Activity Diagram** **24**
4.9 **Component Diagram** **25**
4.10 **Data Flow Diagram** **26-27**
4.11 **E-R Diagram** **28**
4.12 **Deployment Diagram** **29**
4.13 **System Architecture Diagram** **30**

5. **DEVELOPMENT TOOLS** **31**
5.1 **GENERAL** **31**
5.2 **FEATURES OF JAVA** **31**
5.2.1 The Java FrameWork 31
5.2.2 Objectives Of Java 32-33
5.2.3 Java Swing OverView 33-34
5.2.4 Evolution Of Collection FrameWork 34-37
5.3 **CONCLUSION** **37**

6. **IMPLEMENTATION** **38-50**

6.1 **GENERAL** **38-50**
7. **SNAPSHOTS** **51-55**
8. **SOFTWARE TESTING** **56**
8.1 **GENERAL** **56**

8.2 **DEVELOPING METHODOLOGIES** **56**
8.3 **Types Of Tests** **56**
8.3.1 Unit Testing 56
8.3.2 Functional Test 57
8.3.3 System Test 57
8.3.4 Performance Test 57
8.3.5 Integration Testing 58
8.3.6 Acceptance Testing 58
8.3.7 Build The Test Plan 58
9. **APPLICATION** **59**
9.1 **GENERAL** **59**
9.2 **FUTURE ENHANCEMENT** **59**

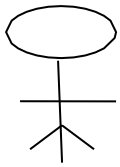
10. **CONCLUSION AND REFERENCE** **60**
10.1 Conclusion 60
10.2 Reference 61-63

LIST OF FIGURES

FIGURE	TITLE	PAGE NUMBER
4.2	Use case diagram	18
4.3	Class diagram	19
4.4	Object Diagram	20
4.5	State-Chart Diagram	21
4.6	Sequence diagram	22
4.7	Collaboration diagram	23
4.8	Activity Diagram	24
4.9	Component diagram	25
4.10	Data flow Diagram	26-27
4.11	E-R Diagram	28
4.12	Deployment Diagram	29
4.13	System Architecture	30

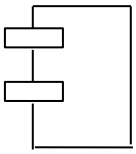
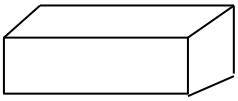
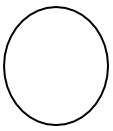
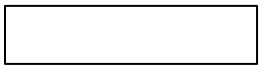
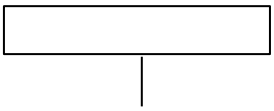
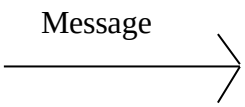
E

LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class	*Class Name* *-attribute* *-attribute* *+ public* *-private*	Represents a collection of similar entities grouped together.
2.	Association	Class A — NAM — Class B Class A — Class B	Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		It aggregates several classes into a single classes.
4.	Aggregation	Class A → Class B Class A → Class B	Interaction between the system and external environment

5.	Relation (uses)	uses	Used for additional process communication.
6.	Relation (extends)	extends	Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State	State	State of the processes.
9.	Initial State		Initial state of the object

10.	Final state		Final state of the object
11.	Control flow		Represents various control flow between the states.
12.	Decision box		Represents decision making process from a constraint
13.	Use case	Uses case	Interact ion between the system and external environment.

14.	Component		Represents physical modules which are a collection of components.
15.	Node		Represents physical modules which are a collection of components.
16.	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or action.
17.	External entity		Represents external entities such as keyboard, sensors, etc.
18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	EXPANSION
1.	DB	Database
2.	JVM	Java Virtual Machine
3.	JSP	Java Server Page
4.	CB	Collective Behaviour
5.	RSS	Ramp secret sharing scheme
6.	JRE	Java Runtime Environment

CHAPTER-1

INTRODUCTION

1.1 GENERAL

With the rapid development of computer technology and the Internet, more and more data are generated by each individual, and all kinds of multimedia information are spread on the Internet in various situations. However, while the Internet greatly facilitates people's production and life, network information security is also facing increasingly severe challenges. How to ensure information security has become a concern of the whole society. Currently, three main methods are being used to protect data: cryptography, watermarking, and steganography. Cryptography involves scrambling secret information to make it unreadable to eavesdroppers. Cryptography not only turns plaintext into ciphertext, but also exposes the importance of secret information, thus arousing the curiosity of attackers. Steganography and watermarking are two important branches of information hiding and they embed data transparently into carrier objects. Watermarking establishes the identity of information to prevent unauthorized use. It embeds the information into digital file in such a way that its removal is hardly possible. The main difference between steganography and watermarking is that watermarking does not necessarily hide the fact that information is transmitted secretly from a third party. Steganography emphasizes the imperceptibility and undetectability of information and requires a considerable capacity for communication efficiency. The object to be protected by digital watermarking is the carrier object, and the main performance required by digital watermarking is robustness.

1.2 OBJECTIVE

This paper begins with a brief introduction of basic steganography methods. Then, the related evaluation indexes of audio steganography are introduced. Next, the paper summarizes the performance of main audio steganography methods. Finally, the paper concludes with recommendations and puts forward suggestions for the coming research focus.

1.3 EXISTING SYSTEM

- Currently, three main methods are being used to protect data: cryptography, watermarking, and steganography.
- All the three approaches are very traditional approaches and they are having their own issues.

1.3.1 EXISTING SYSTEM DISADVANTAGES

- Prone to attacks.
- Heavy code for public and primary keys.
- High maintenance cost

1.3.2 LITERATURE SURVEY

Title: Information hiding using audio steganography survey

Author: Jayaram P, Ranganatha H R, Anupama H S.

Year: 2011.

Description: Today's large demand of internet applications requires data to be transmitted in a secure manner. Data transmission in public communication system is not secure because of interception and improper manipulation by eavesdropper. So the attractive solution for this problem is Steganography, which is the art and science of writing hidden messages in such a way that no one, apart from the sender and intend recipient, suspects the existence of the message, a form of security through obscurity. Audio steganography is the scheme of hiding the existence of secret information by concealing it into another medium such as audio file. In this paper we mainly discuss different types of audio steganographic methods, advantages and disadvantages.

Title: A genetic-algorithm based approach for audio Steganography.

Author: Zamani M, Manaf A A, Ahmad R B, et al.

Year: 2009.

Description: In this paper, we present a novel, principled approach to resolve the remained problems of substitution technique of audio steganography. Using the proposed genetic algorithm, message bits are embedded into multiple, vague and higher LSB layers, resulting in increased robustness. The robustness specially would be increased against those intentional attacks which try to reveal the hidden message and also some unintentional attacks like noise addition as well.

Title: Genetic algorithm for fragile audio watermarking

Author: Zamani M, Manaf A B A.

Year: 2011.

Description: A novel genetic-concept-based algorithm is proposed for fragile audio watermarking to reduce the distortion of least significant bits substitution and consequently improve the peak signal-to-noise ratio (PSNR) and increase the payload of results. The result of testing shows that in comparison with ordinary substitution techniques and other presented techniques, the payload is considerably increased and PSNR (as an indicator of imperceptibility) is noticeably raised.

Title: A view on latest audio Steganography techniques

Author: Djebbar F, Ayad B, Hamam H, et al.

Year: 2011

Description: Steganography has been proposed as a new alternative technique to enforce data security. Lately, novel and versatile audio steganographic methods have been proposed. A perfect audio Steganographic technique aim at embedding data in an imperceptible, robust and secure way and then extracting it by authorized people. Hence, up to date the main challenge in digital audio steganography is to obtain robust high capacity steganographic systems. Leaning towards designing a system that ensures high capacity or robustness and security of embedded data has led to great diversity in the existing steganographic techniques. In this paper, we present a current state of art literature in digital audio steganographic techniques. We explore their potentials and limitations to ensure secure communication. A comparison and an evaluation for the reviewed techniques is also presented in this paper.

Title: Video steganography: a comprehensive review.

Author: M. Zhou and A. Mockus

Year: 2015**.**

Description: Steganography is the art and science of secret communication, concealing the very existence of a communication. Modern cover types can take many forms such as text documents, audio tracks, digital images, and video streams. Extensive research has been done on image steganography in the previous decade due to their popularity on the internet. Nowadays, video files are drawing much more attention. They are transmitted more and more frequent on internet websites such as Facebook and YouTube imposing a larger practical significance on video steganography. Information hiding in video has a variety of techniques, each of which has its strengths and weaknesses. This paper intends to provide an up-to-date comprehensive review on the various video steganographic methods found in the literature in the last 5 years. Furthermore, since security and robustness are very important issues in designing a good steganographic algorithm, some relevant attacks and steganalysis techniques are also surveyed. The paper concludes with recommendations and good practices drawn from the reviewed techniques.

Title: Twenty years of digital audio watermarking a comprehensive review.

Author: Hua G, Huang J, Shi Y Q, et al.

Year: 2016

Description: Digital audio watermarking is an important technique to secure and authenticate audio media. This paper provides a comprehensive review of the twenty years' research and development works for digital audio watermarking, based on an exhaustive literature survey and careful selections of representative solutions. We generally classify the existing designs into time domain and transform domain methods, and relate all the reviewed works using two generic watermark embedding equations in the two domains. The most important designing criteria, i.e., imperceptibility and robustness, are thoroughly reviewed. For imperceptibility, the existing measurement and control approaches are classified into heuristic and analytical types, followed by intensive analysis and discussions. Then, we investigate the robustness of the existing solutions against a wide range of critical attacks categorized into basic, desynchronization, and replacement attacks, respectively. This reveals current challenges **in**

Title: A recent review of MP3 based Steganography methods.

Author: Bazyar M, Sudirman R.

Year: 2014

Description:

Steganography, the idea of hiding messages and data within other pieces of data, can be useful in many real world applications alongside encryption and other code-writing methods. Data hiding methods are a new kind of technology in secret communication. Multimedia objects like audio are the most used in today's information hiding systems. Audio file due to its high degree of redundancy and high data transmission rate can create a good hiding medium. Several formats such as MP3 have been used in audio data steganography. Varieties of methods have been established for embedding data in digital audio. In this article, in addition to more emphasis on MPEG-1 layer III (MP3) steganography methods with thorough information about weaknesses and strengths of these techniques, we present a complete study of audio file steganography approaches.

1.4 Proposed System

- There are many carriers of steganography, including image, audio, video, text and other multimedia files.
- At present, there are many researches on image steganography.
- Compared with image, audio steganography is less studied. Since human auditory system (HVS) is more sensitive than human visual system (HAS), the change of audio signal is more likely to cause human sensory perception, which also puts forward higher requirements for audio steganography technology.

1.4.1 Proposed System Advantage

- Easy to use.
- Hard for attackers to guess the password.
- Low Maintenance cost
- Audio based Steganography has the potential to conceal more information.
- The flexibility of audio steganography is makes it very potentially powerful.

CHAPTER 2

PROJECT DESCRIPTION

2.1 GENERAL

There are many carriers of steganography, including image, audio, video, text and other multimedia files. At present, there are many researches on image steganography. Compared with image, audio steganography is less studied. Audio steganography methods can be divided into three categories: time domain, frequency domain and wavelet domain . This section introduces them respectively.

2.2 METHODOLOGIES

2.2.1 MODULES NAME

Modules Name:

This project having the following modules:

- **Audio Steganography Evaluation**
- **User**
- **Parity Coding**

2.2.2 MODULES EXPLANATION

- **Audio Steganography Evaluation**

The evaluation indexes of audio steganography include steganography capacity, undetectability, imperceptibility, robustness and security. Security is generally ensured by encryption, that is, secret messages are encrypted to form binary bit streams before embedding. Undetectability is mainly used in some audio watermarking technologies. For audio steganography, the three indexes of steganography capacity, imperceptibility and robustness are more important. Imperceptibility refers to the performance that the original audio still retains a certain quality without causing human ears to detect it on the basis of embedding secret information.

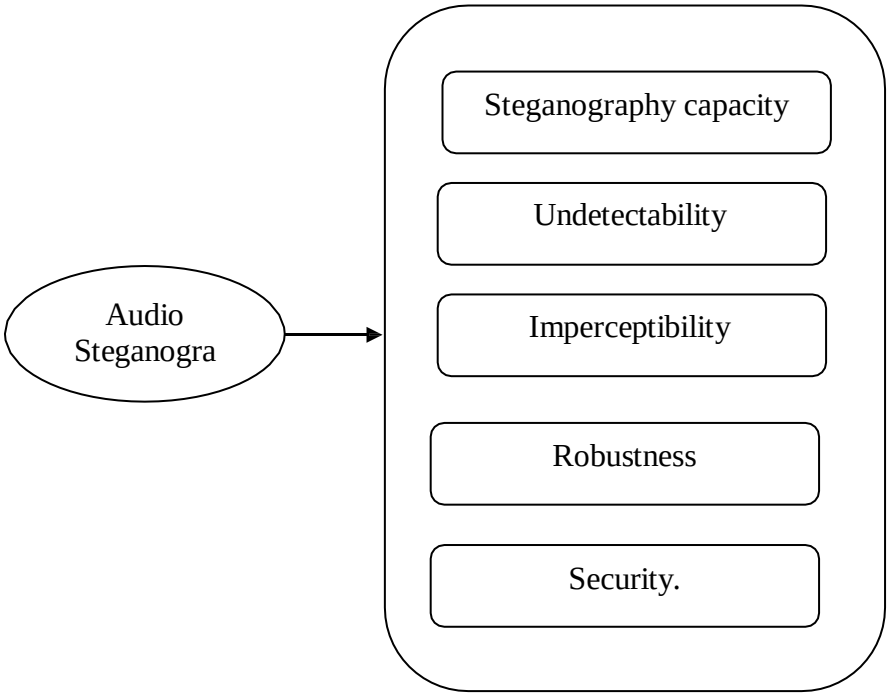
- **User**
  - User must have to select audio file of type .au format.
  - User have to write text message or secret message for embedding it in audio file.
  - User must have to select a audio file for encoding message.
  - User must have to select a stego-audio file for decoding message.
  - User must have to give key for encryption and for decryption**.**

  - **Parity Coding**

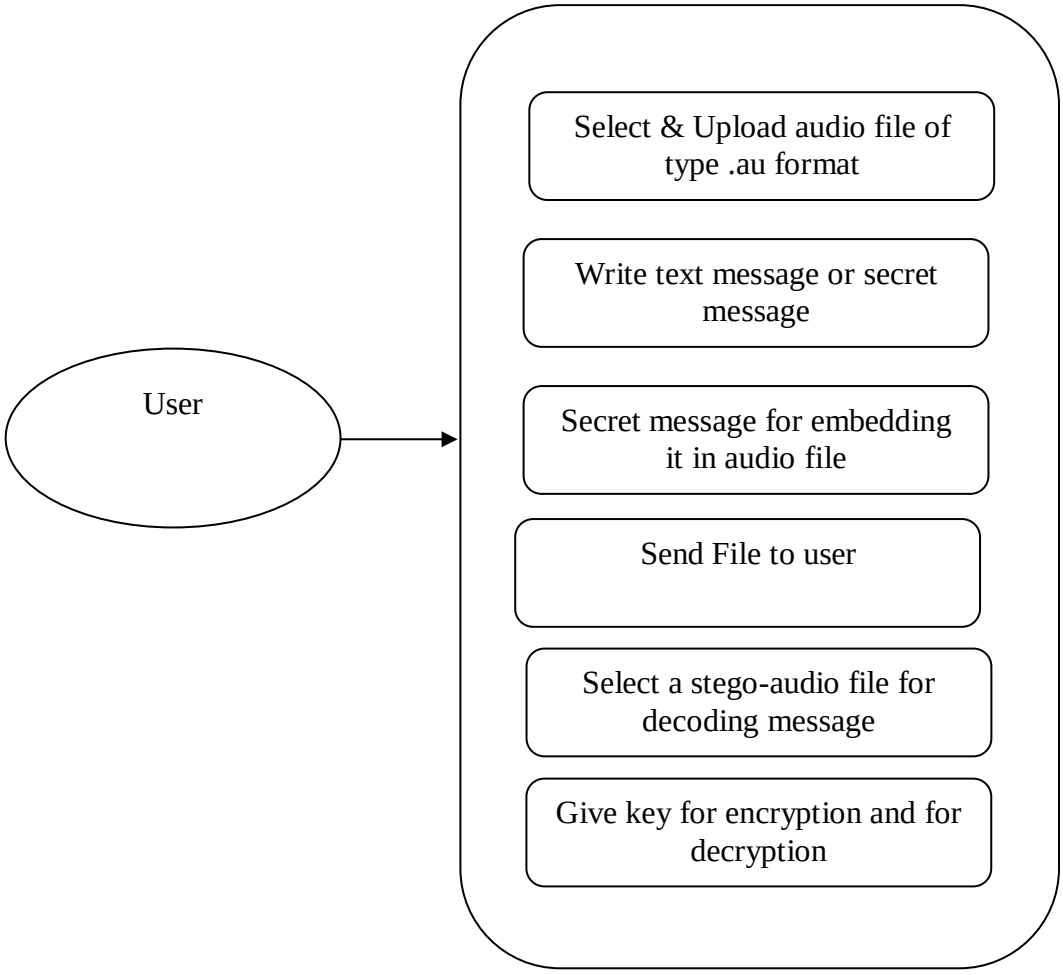
    Parity coding method decomposes the original signal into different regions of samples and hides the secret message in the parity bits of the sample region. If the secret message matches the check bit, proceed. Otherwise, reverse the LSB bit of a sample in this region. The concept of this technique. With parity coding, the sender has more choices for the modified secret bits.

2.2.3 MODULE DIAGRAM

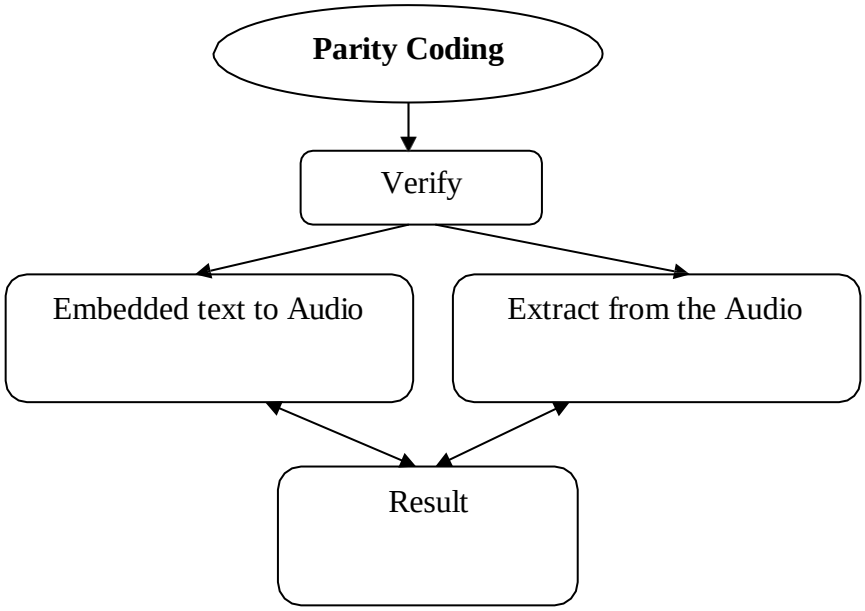
- **Audio Steganography Evaluation**



- **User**


User
Select & Upload audio file of type .au format
Write text message or secret message
Secret message for embedding it in audio file
Send File to user
Select a stego-audio file for decoding message
Give key for encryption and for decryption

- **Parity Coding**



2.2.4 GIVEN INPUT EXPECTED OUTPUT:

- **Audio Steganography Evaluation**

Input : Providing Audio evaluation

Output: Security is generally ensured by encryption, that is, secret messages are encrypted to form binary bit streams before embedding.

- **User**

Input : User audio steganography

Output : User Select & upload text messages & audio embedding files.

- **Parity Coding**

Input : Complete verification

Output : parity coding method decomposes the original signal into different regions of samples and hides the secret message in the parity bits of the sample region.

2.3 TECHNIQUE USED OR ALGORITHM USED

- **AUDIO STEGANOGRAPHY**

  Comparison Criteria The evaluation indexes of audio steganography include steganography capacity, undetectability, imperceptibility, robustness and security. Security is generally ensured by encryption, that is, secret messages are encrypted to form binary bit streams before embedding. Undetectability is mainly used in some audio watermarking technologies. For audio steganography, the three indexes of Steganography capacity, imperceptibility and robustness are more important .

- Audio based Steganography has the potential to conceal more information.
- The flexibility of audio steganography is makes it very potentially powerful.
- Many attacks that are malicious against image Steganography algorithms (e.g. geometrical distortions, spatial scaling, etc.) cannot be implemented against audio Steganography schemes. Consequently, embedding information into audio seems more secure due to less steganalysis techniques for attacking to audio.

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 GENERAL

These are the requirements for doing the project. Without using these tools and software's we can't do the project. So we have two requirements to do the project. They are

- 1. Hardware Requirements.
- 2. Software Requirements.

3.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It should what the system do and not how it should be implemented.

HARDWARE

- PROCESSOR : PENTIUM IV 2.6 GHz, Intel Core 2 Duo.
- RAM : 512 MB DD RAM
- MONITOR : 15" COLOR
- HARD DISK : 40 GB

3.3 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the teams and tracking the team's progress throughout the development activity.

SOFTWARE

- Front End : Java
- Operating System : Windows 7
- IDE : Eclipse

3.4 FUNCTIONAL REQUIREMENTS

A functional requirement defines a function of a software-system or its component. A function is described as a set of inputs, the behaviour, and outputs. Security is generally ensured by encryption, that is, secret messages are encrypted to form binary bit streams before embedding. Undetectability is mainly used in some audio watermarking technologies. For audio steganography, the three indexes of steganography capacity, imperceptibility and robustness are more important. Imperceptibility refers to the performance that the original audio still retains a certain quality without causing human ears to detect it on the basis of embedding secret information.

3.5 NON-FUNCTIONAL REQUIREMENTS

The major non-functional Requirements of the system are as follows

- **Usability**

The system is designed with completely automated process hence there is no or less user intervention.

- **Reliability**

The system is more reliable because of the qualities that are inherited from the chosen platform java. The code built by using java is more reliable.

- **Performance**

This system is developing in the high level languages and using the advanced front-end and back-end technologies it will give response to the end user on client system with in very less time.

- **Supportability**

The system is designed to be the cross platform supportable. The system is supported on a wide range of hardware and any software platform, which is having JVM, built into the system.

- **Implementation**

The system is implemented in web environment using struts framework. The apache tomcat is used as the web server and windows xp professional is used as the platform. Interface the user interface is based on Struts provides HTML Tag

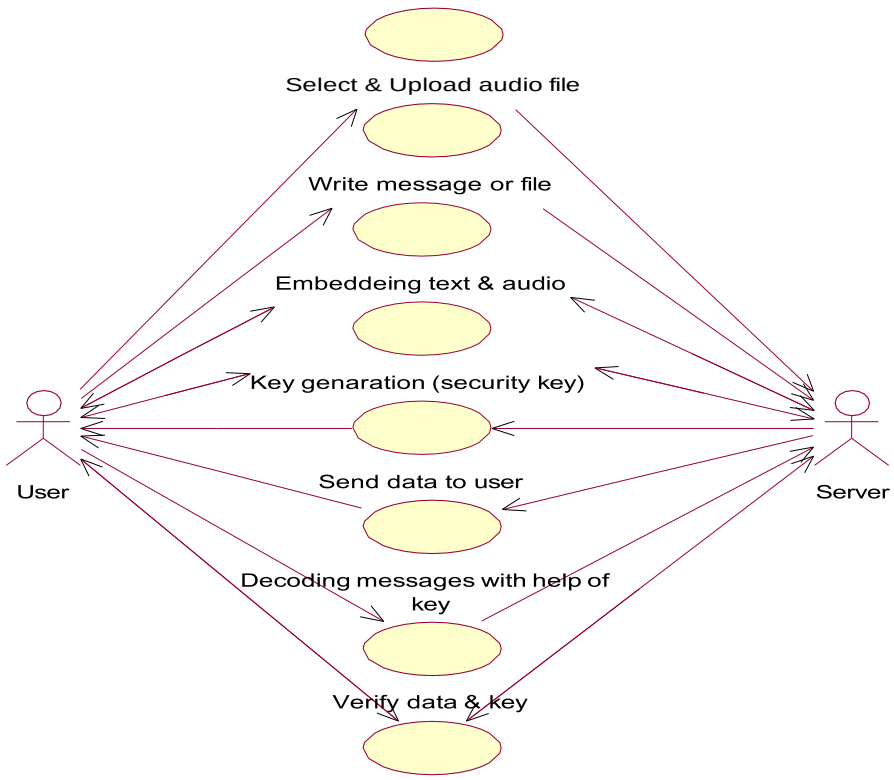
CHAPTER 4

DESIGN ENGINEERING

4.1 GENERAL

Design Engineering deals with the various UML [Unified Modelling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering. Design is the means to accurately translate customer requirements into finished product.

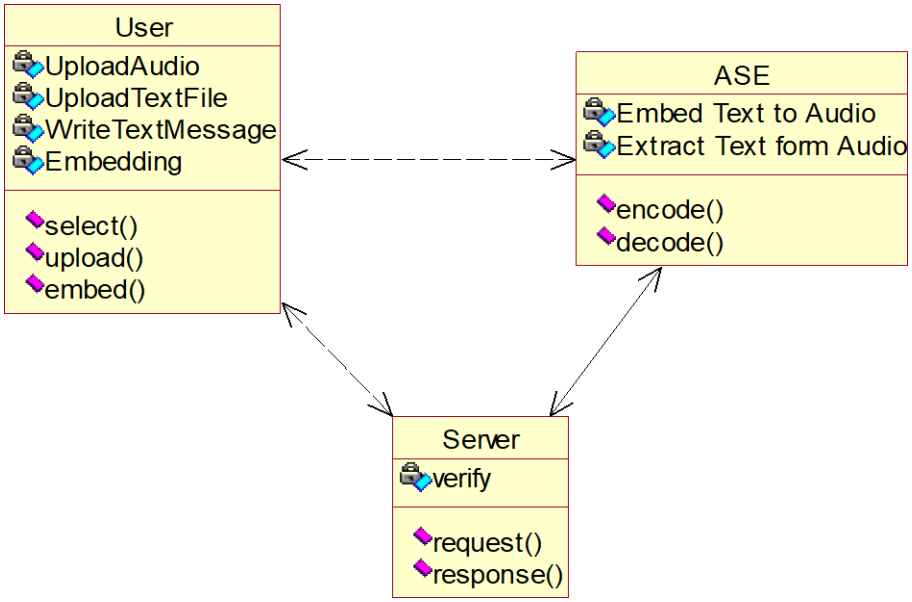
4.2 Use Case Diagram



EXPLANATION:

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. The above diagram consists of user as actor. Each will play a certain role to achieve the concept .

4.3 Class Diagram

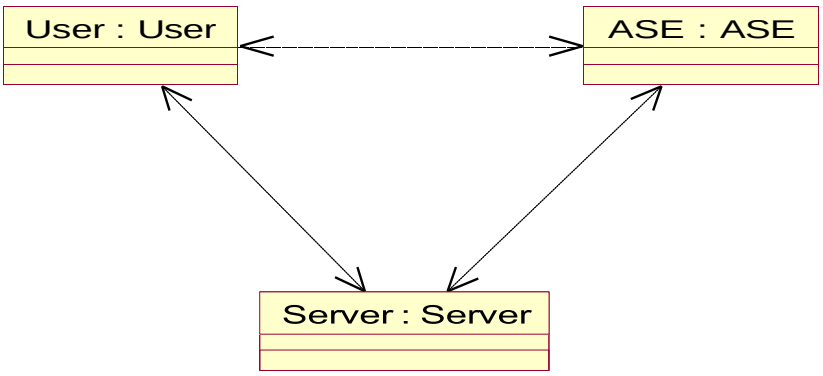


EXPLANATION

In this class diagram represents how the classes with attributes and methods are linked together to perform the verification with security. From the above diagram shown the various classes involved in our project.

.

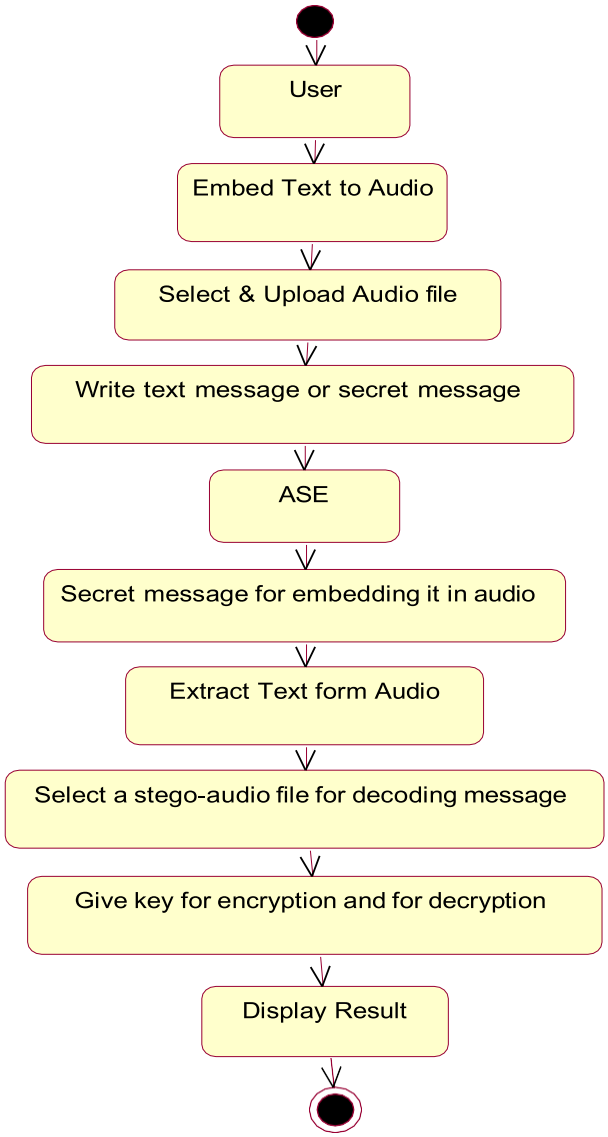
4.4 OBJECT DIAGRAM



EXPLANATION:

In the above diagram tells about the flow of objects between the classes. It is a diagram that shows a complete or partial view of the structure of a modeled system. In this object diagram represents how the classes with attributes and methods are linked together to perform the verification with security.

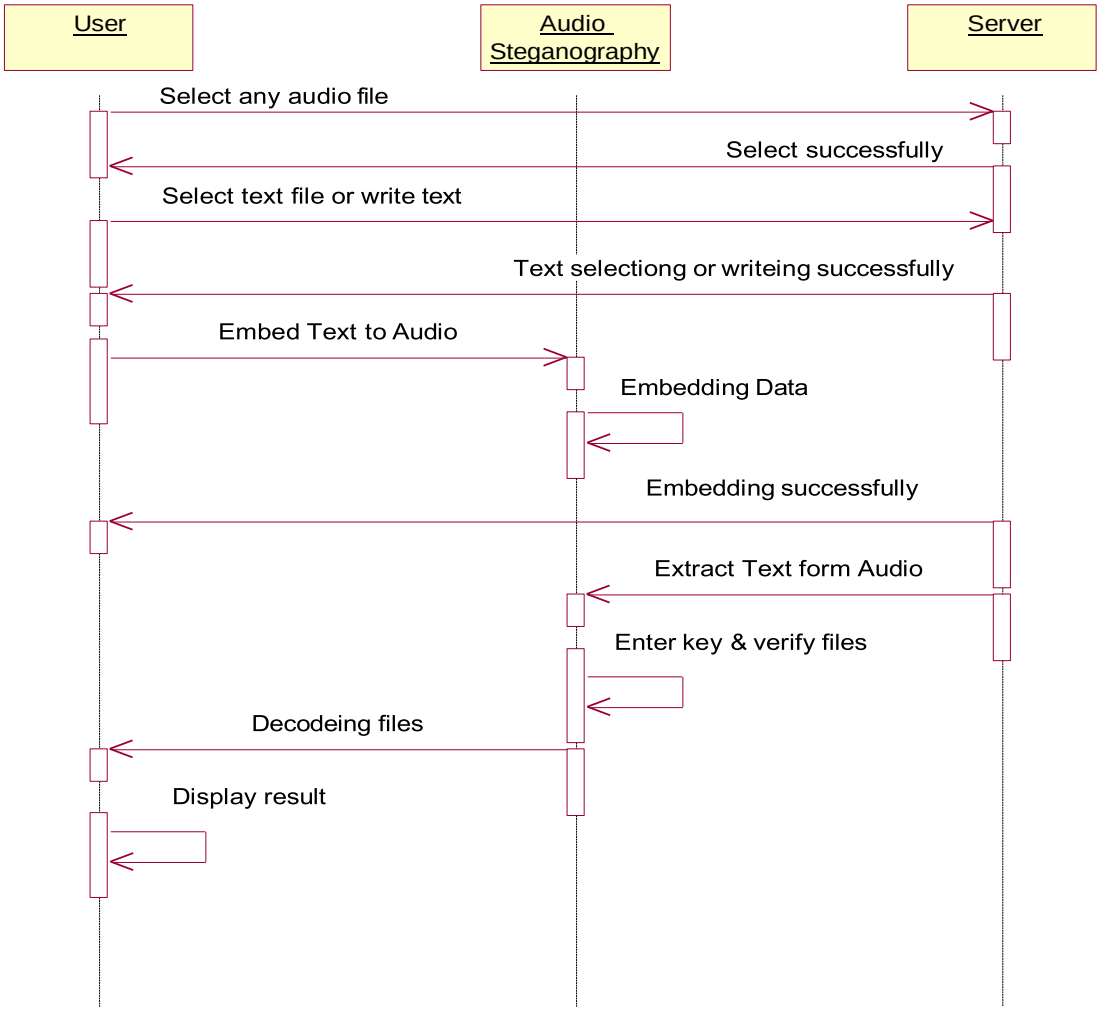
4.5 STATE CHART DIAGRAM



EXPLANATION:

State diagram are a loosely defined diagram to show workflows of stepwise activities and actions, with support for choice, iteration and concurrency. State diagrams require that the system described is composed of a finite number of states; sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics

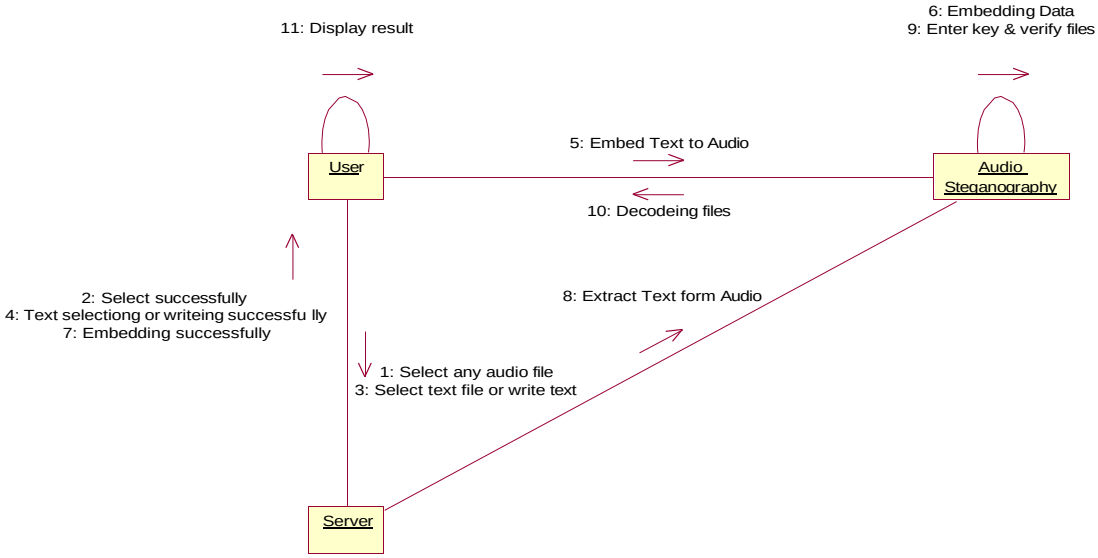
4.6 Sequence Diagram



EXPLANATION:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

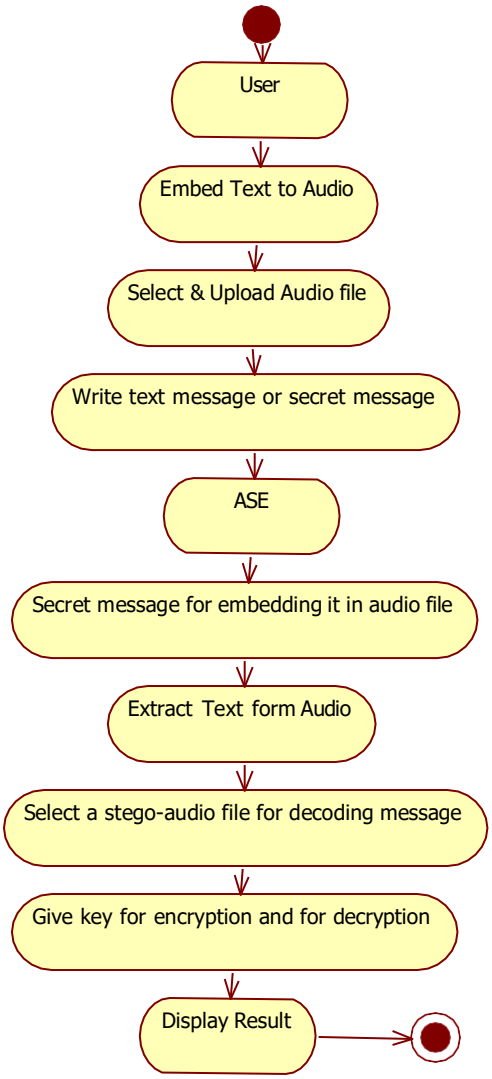
4.7 Collaboration Diagram



EXPLANATION:

A collaboration diagram, also called a communication diagram or interaction diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML). The concept is more than a decade old although it has been refined as modeling paradigms have evolved.

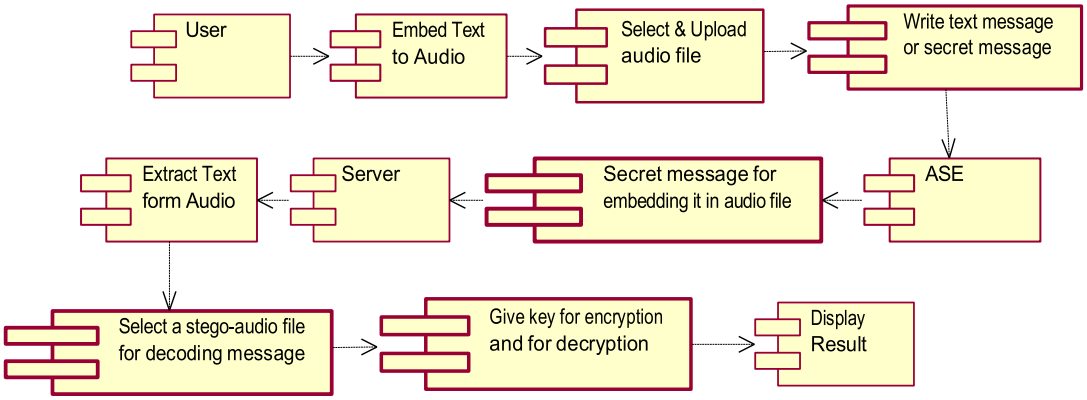
4.8 Activity Diagram



EXPLANATION:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

4.9 COMPONENT DIAGRAM

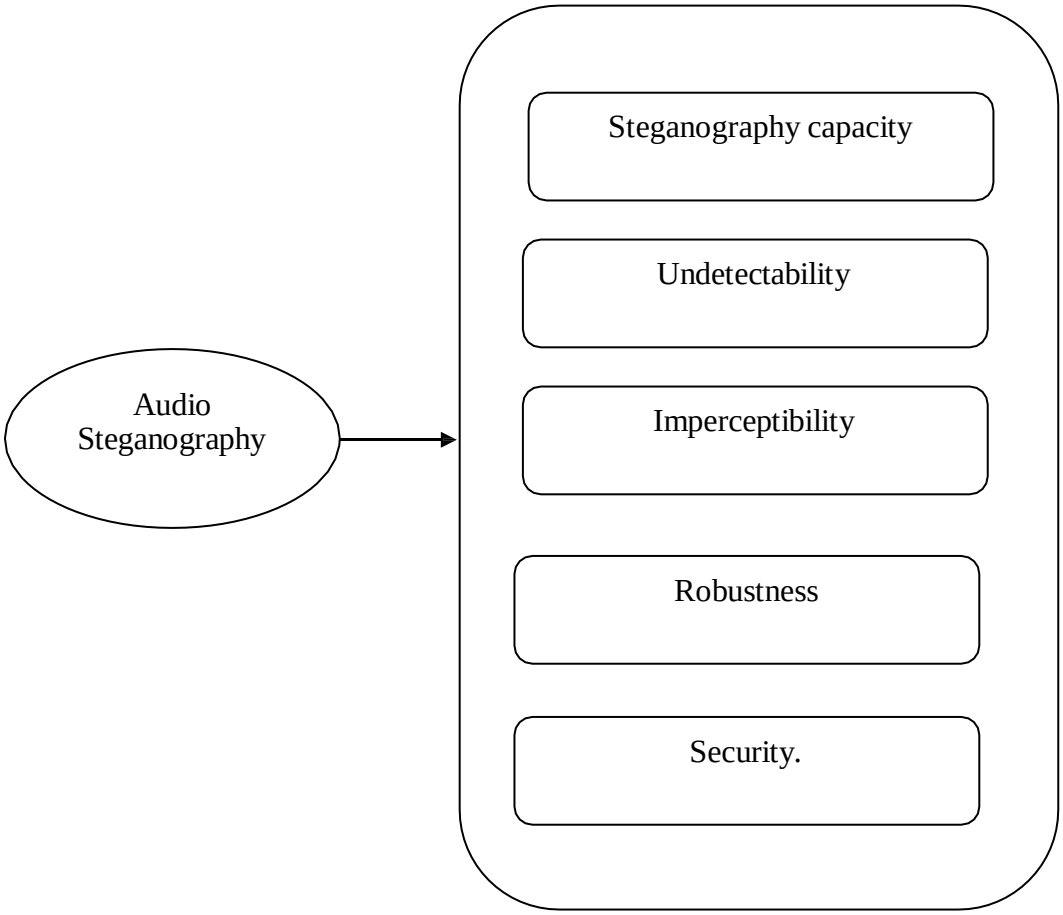


EXPLANATION:

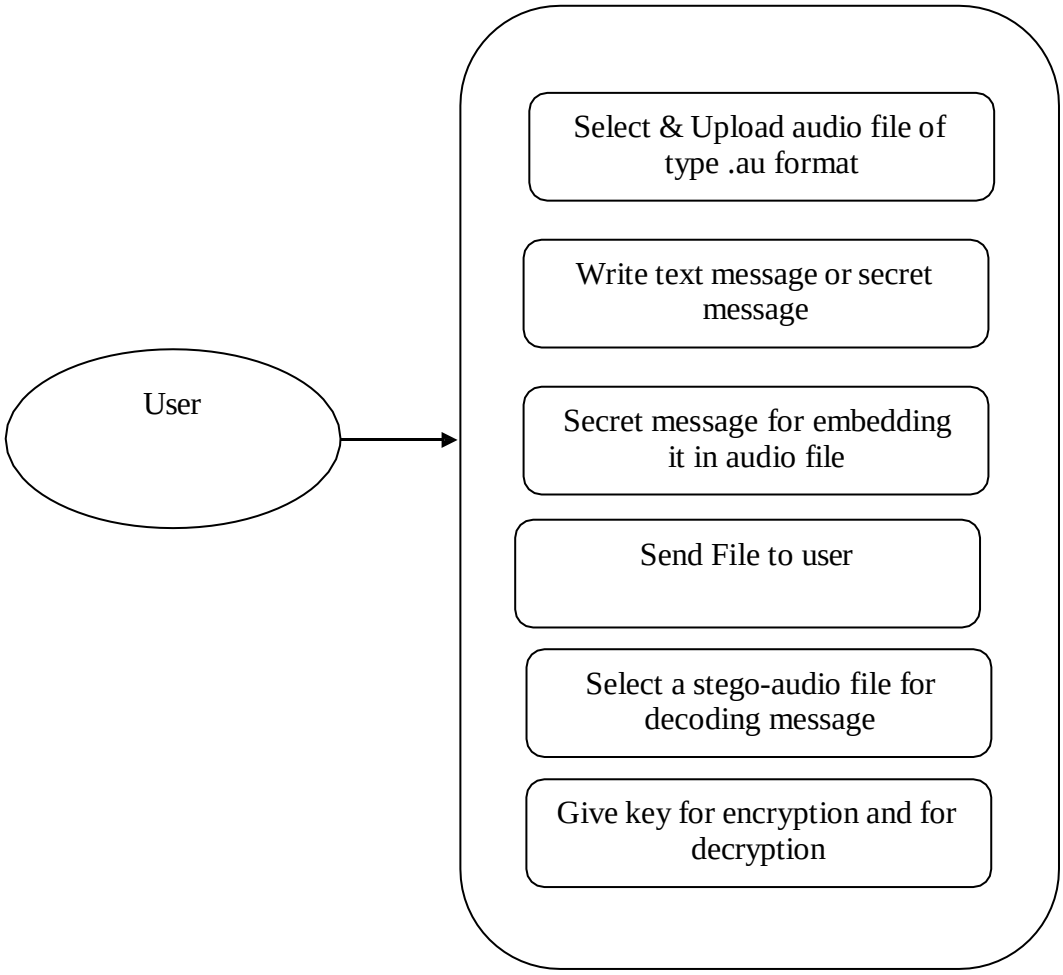
In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems. User gives main query and it converted into sub queries and sends through data dissemination to data aggregators. Results are to be showed to user by data aggregators. All boxes are components and arrow indicates dependencies.

4.10 DATA FLOW DIAGRAM

Level 0:



Level 1:

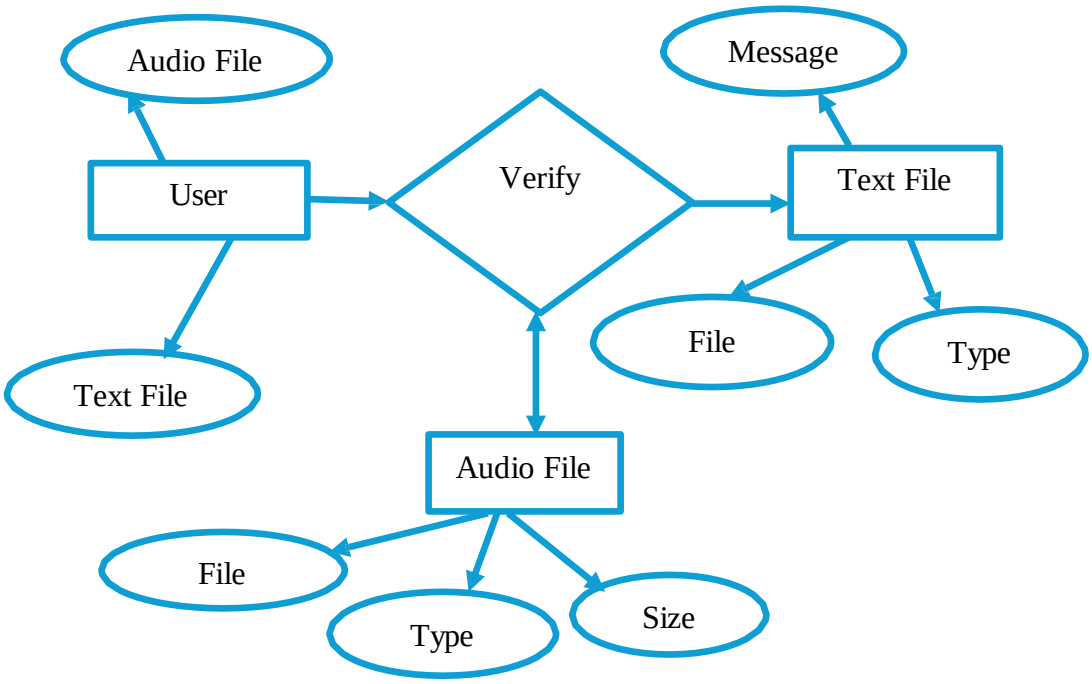


EXPLANATION:

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modeling its process aspects. Often they are a preliminary step used to create an overview of the system which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design).

A DFD shows what kinds of data will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of processes, or information about whether processes will operate in sequence or in parallel.

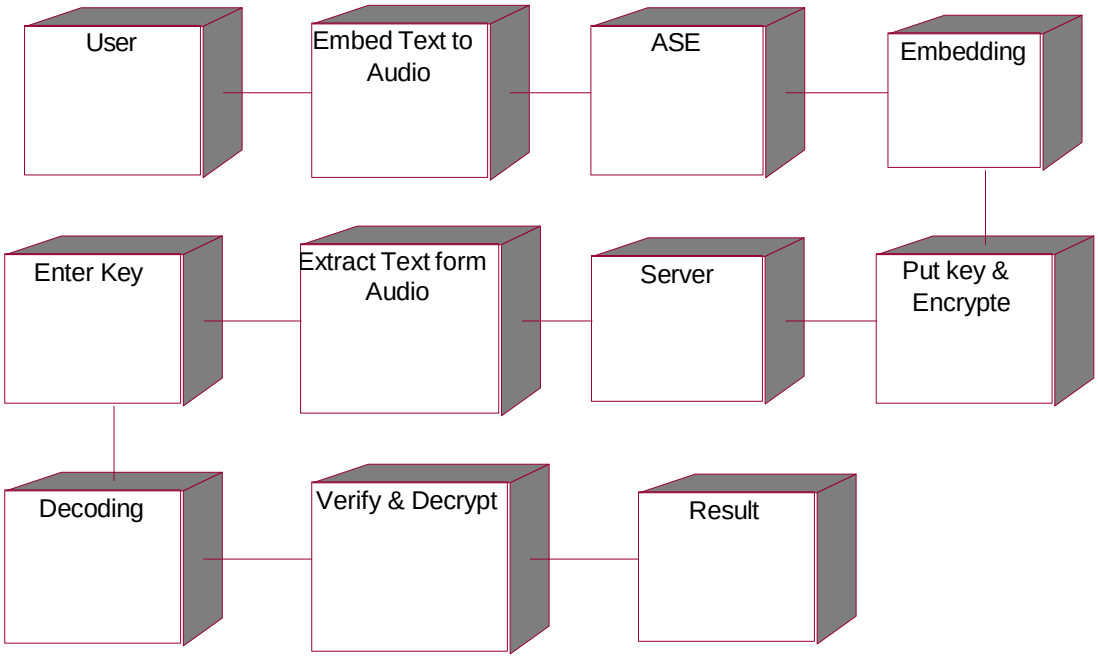
4.11 E-R Diagram:



EXPLANATION:

Entity-Relationship Model (ERM) is an abstract and conceptual representation of data. Entity-relationship modeling is a database modeling method, used to produce a type of conceptual schema or semantic data model of a system, often a relational database.

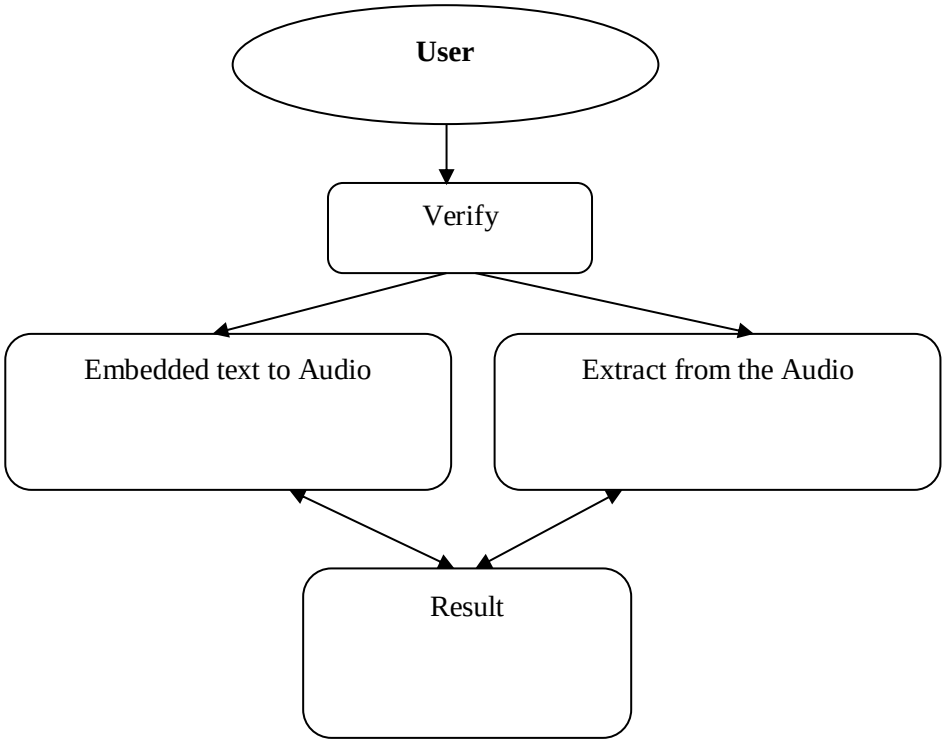
4.12 Deployment Diagram:



EXPLANATION:

In the Unified Modeling Language, a deployment diagram depicts how deployment are wired together to form larger deployed and or software systems. They are used to illustrate the structure of arbitrarily complex systems. User gives main query and it converted into sub queries and sends through data dissemination to data aggregators. Results are to be showed to user by data aggregators. All boxes are components and arrow indicates dependencies.

4.13 System Architecture



CHAPTER 5

DEVELOPMENT TOOLS

5.1 GENERAL

This chapter is about the software language and the tools used in the development of the project. The platform used here is JAVA. The Primary languages are JAVA, J2EE and J2ME. In this project J2EE is chosen for implementation.

5.2 FEATURES OF JAVA

5.2.1 THE JAVA FRAMEWORK

Java is a programming language originally developed by James Gosling at Microsystems and released in 1995 as a core component of Sun Microsystems' Java platform. The language derives much of its syntax from C and C++ but has a simpler object model and fewer low-level facilities. Java applications are typically compiled to byte code that can run on any Java Virtual Machine (JVM) regardless of computer architecture. Java is general-purpose, concurrent, class-based, and object-oriented, and is specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere".

Java is considered by many as one of the most influential programming languages of the 20th century, and is widely used from application software to web applications the java framework is a new platform independent that simplifies application development internet. Java technology's versatility, efficiency, platform portability, and security make it the ideal technology for network computing. From laptops to datacenters, game consoles to scientific supercomputers, cell phones to the Internet, Java is everywhere!

5.2.2 OBJECTIVES OF JAVA

To see places of Java in Action in our daily life, explore java.com.

Why Software Developers Choose Java

Java has been tested, refined, extended, and proven by a dedicated community. And numbering more than 6.5 million developers, it's the largest and most active on the planet. With its versatility, efficiency, and portability, Java has become invaluable to developers by enabling them to:

- Write software on one platform and run it on virtually any other platform
- Create programs to run within a Web browser and Web services
- Develop server-side applications for online forums, stores, polls, HTML forms processing, and more
- Combine applications or services using the Java language to create highly customized applications or services
- Write powerful and efficient applications for mobile phones, remote processors, low-cost consumer products, and practically any other device with a digital heartbeat

Some Ways Software Developers Learn Java

Today, many colleges and universities offer courses in programming for the Java platform. In addition, developers can also enhance their Java programming skills by reading Sun's java.sun.com Web site, subscribing to Java technology-focused newsletters, using the Java Tutorial and the New to Java Programming Center, and signing up for Web, virtual, or instructor-led courses.

Object Oriented

To be an Object Oriented language, any language must follow at least the four characteristics.

1. Inheritance : It is the process of creating the new classes and using the behavior of the existing classes by extending them just to reuse the existing code and adding addition a features as needed.

2. Encapsulation: It is the mechanism of combining the information and providing the abstraction.

3. Polymorphism: As the name suggest one name multiple form, Polymorphism is the way of providing the different functionality by the functions having the same name based on the signatures of the methods.

4. Dynamic binding: Sometimes we don't have the knowledge of objects about their specific types while writing our code. It is the way of providing the maximum functionality to a program about the specific type at runtime

5.2.3 JAVA SWING OVERVIEW

Abstract Window Toolkit (AWT) is cross-platform

Swing[1] provides many controls and widgets to build user interfaces with. Swing class names typically begin with a J such as JButton, JList, JFrame. This is mainly to differentiate them from their AWT counterparts and in general is one-to-one replacements. Swing is built on the concept of Lightweight components vs AWT and SWT's concept of Heavyweight components. The difference between the two is that the Lightweight components are rendered (drawn) using purely Java code, such as draw Line and draw Image, whereas Heavyweight components use the native operating system to render the components.

Some components in Swing are actually heavyweight components. The top-level classes and any derived from them are heavyweight as they extend the AWT versions. This is needed because at the root of the UI, the parent windows need to be provided by the OS. These top-level classes include `JWindow`, `JFrame`, `JDialog` and `JApplet`. All Swing components to be rendered to the screen must be able to trace their way to a root window of one of those classes.

Note: It generally it is not a good idea to mix heavyweight components with lightweight components (other than as previously mentioned) as you will encounter layering issues,

e.g., a lightweight component that should appear "on top" ends up being obscured by a heavyweight component. The few exceptions to this include using heavyweight components as the root pane and for popup windows. Generally speaking, heavyweight components will render on top of lightweight components and will not be consistent with the look and feel being used in Swing. There are exceptions, but that is an advanced topic. The truly adventurous may want to consider reading this article from Sun on mixing heavyweight and lightweight components.

5.2.4 Evolution of Collection Framework:

Almost all collections in Java are derived from the **java.util.Collection** interface. Collection defines the basic parts of all collections. The interface states the add() and remove() methods for adding to and removing from a collection respectively. Also required is the toArray() method, which converts the collection into a simple array of all the elements in the collection. Finally, the contains() method checks if a specified element is in the collection. The Collection interface is a sub interface of **java.util.Iterable**, so the iterator() method is also provided. All collections have an iterator that goes through all of the elements in the collection. Additionally, Collection is a generic. Any collection can be written to store any class. For example, Collection<String> can hold strings, and the elements from the collection can be used as strings without any casting required.

There are three main types of collections:

- Lists: always ordered, may contain duplicates and can be handled the same way as usual arrays
- Sets: cannot contain duplicates and provide random access to their elements
- Maps: connect unique keys with values, provide random access to its keys and may host duplicate values

LIST

Lists are implemented in the JCF via the java.util.List interface. It defines a list as essentially a more flexible version of an array. Elements have a specific order, and duplicate elements are allowed. Elements can be placed in a specific position. They can also be searched for within the list. Two concrete classes implement List. The first is java.util.ArrayList, which implements the list as an array. Whenever functions specific to a list are required, the class moves the elements around within the array in order to do it. The other implementation is java.util.LinkedList. This class stores the elements in nodes that each have a pointer to the previous and next nodes in the list. The list can be traversed by following the pointers, and elements can be added or removed simply by changing the pointers around to place the node in its proper place.

SET:

Java's java.util.Set interface defines the set. A set can't have any duplicate elements in it. Additionally, the set has no set order. As such, elements can't be found by index. Set is implemented by java.util.HashSet,java.util.LinkedHashSet, and java.util.TreeSet. HashSet uses a hash table. More specifically, it uses a java.util.HashMap to store the hashes and elements and to prevent duplicates. Java.util.LinkedHashSet extends this by creating a doubly linked list that links all of the elements by their insertion order. This ensures that the iteration order over the set is predictable. java.util.TreeSet uses a red-black tree implemented by a java.util.TreeMap. The red-black tree makes sure that there are no duplicates. Additionally, it allows Tree Set to implement java.util.SortedSet.

The java.util.Set interface is extended by the java.util.SortedSet interface. Unlike a regular set, the elements in a sorted set are sorted, either by the element's compareTo() method, or a method provided to the constructor of the sorted set. The first and last elements of the sorted set can be retrieved, and subsets can be created via minimum and maximum values, as well as beginning or ending at the beginning or ending of the sorted set. The SortedSet interface is implemented by java.util.TreeSet

java.util.SortedSet is extended further via the java.util.NavigableSet interface. It's similar to SortedSet, but there are a few additional methods. The floor(), ceiling(), lower(), and higher() methods find an element in the set that's close to the parameter. Additionally, a descending iterator over the items in the set is provided. As with SortedSet, java.util.TreeSet implements NavigableSet.

MAP:

Maps are defined by the java.util.Map interface in Java. Maps are simple data structures that associate a key with a value. The element is the value. This lets the map be very flexible. If the key is the hash code of the element, the map is essentially a set. If it's just an increasing number, it becomes a list. Maps are implemented by java.util.HashMap, java.util.LinkedHashMap, and java.util.TreeMap. HashMap uses a hash table. The hashes of the keys are used to find the values in various buckets. LinkedHashMap extends this by creating a doubly linked list between the elements. This allows the elements to be accessed in the order in which they were inserted into the map. TreeMap, in contrast to HashMap and LinkedHashMap, uses a red-black tree. The keys are used as the values for the nodes in the tree, and the nodes point to the values in the map

Thread:

Simply put, a *thread* is a program's path of execution. Most programs written today run as a single thread, causing problems when multiple events or actions need to occur at the same time. Let's say, for example, a program is not capable of drawing pictures while reading keystrokes. The program must give its full attention to the keyboard input lacking the ability to handle more than one event at a time. The ideal solution to this problem is the seamless execution of two or more sections of a program at the same time.

Creating threads

Java's creators have graciously designed two ways of creating threads: implementing an interface and extending a class. Extending a class is the way Java inherits methods and variables from a parent class. In this case, one can only extend or inherit from a single parent class. This limitation within Java can be overcome by implementing interfaces, which is the most common way to create threads.

Interfaces provide a way for programmers to lay the groundwork of a class. They are used to design the requirements for a set of classes to implement. The interface sets everything up, and the class or classes that implement the interface do all the work. The different set of classes that implement the interface have to follow the same rules.

5.5 Conclusion

Swing's high level of flexibility is reflected in its inherent ability to override the native host operating system (OS)'s GUI controls for displaying itself. Swing "paints" its controls using the Java 2D APIs, rather than calling a native user interface toolkit. The Java thread scheduler is very simple. All threads have a priority value which can be changed dynamically by calls to the threads setPriority() method . Implementing the above concepts in our project to do the efficient work among the Server.

CHAPTER 6

IMPLEMENTATION

6.1 GENERAL CODING:

```
<%@page import="java.util.List"%>
<%@page import="com.dao.UserDAO"%><%
        String loginstatus = "";
        if (session != null) {
           loginstatus = (String) session.getAttribute("status");
              if (!loginstatus.equals("login")) {
                 response.sendRedirect("login.jsp?msg='you are logged out,plz login first'");
           }
        }
%>
<html>
    <%
        String c_user =(String)session.getAttribute("user");
        UserDAO dao =new UserDAO();

    %>
  <head>
  <title>Friends</title>
    <style type="text/css">
        body{
            margin: 0px;
            background-color: black;
            height: 400px;
        }
        .ow {
                overflow-wrap: break-word;
                word-wrap: break-word;
                hyphens: auto;
        }
        .nav{
            font-size: 30px;
            display: inline;
            float: right;
            margin-right:50px;
        }
        .nav:hover{
            text-decoration: underline;
        }
        a{
            color: white;
```

```
            text-decoration: none;
        }
        .req{
            border-bottom:1px solid gray;
            padding: 10px;
        }
        .breq{
            border-radius:5px;
        float: right;
background: lime;
        }
    </style>
  </head>
  <body>
    <div style="width: 99.6%;height: 80px; color: white;">
        <p class="nav"><a href="logout" onclick="return
confirm('Continue!')">Logout</a></p>
        <p class="nav"><a href="user_key.jsp">Key</a></p>
        <p class="nav"><a href="user_friends.jsp"  style="text-decoration:
underline;">Friends</a></p>
        <p class="nav"><a href="user_send.jsp">Send Message</a></p>
        <p class="nav"><a href="user_msg.jsp">Messages</a></p>
        <p class="nav"><a href="users_req.jsp">Requests</a></p>
        <p class="nav"><a href="admin_home.jsp">Home</a></p>
    </div>
    <span style="float: right;">Welcome <%=session.getAttribute("user") %></span>
    <h1 style="color: white;margin:0px;margin-top:70px; font-family: cursive;font-size:70px;
text-align: center">Audio SteganoGraphy</h1>
    <h3 style="text-align: center;color: white;">The Art Of Being Invisible</h3>
    <div style="width: 700px; height: 400px; border: 2px solid white; margin-left:
30px;margin-top: 00px; float: left;">
        <img alt="" src="images/home.jpg" height="400" width="700">
    </div>
    <div style="width: 300px;color:blue; height: 400px;background:lightyellow;margin-
right:30px; border: 2px solid white;float: right;">
        <h2 style="text-align: center;text-decoration: underline;">Friends</h2>
        <div style="width: 95%;margin-left:6px;margin-right:6px; height: 82%;border: 1px
solid gray;overflow: auto;">
            <%
                List<String> list1 =dao.getFrnds(c_user);
                if(list1.size()>0){
                    for(String s:list1){
                        if(!s.equals(c_user)){
            %>
                            <div class="req">
                                <%=s %>
                                <button id="<%=s%>" class="breq"
onclick="window.location.href='profile.jsp?who=<%=s %>'">View Profile</button>
```

```
                    </div>
        <%
                }
            }
          }
        %>
      </div>
    </div>
  </body>
</html>


<%@page import="java.util.List"%>
<%@page import="com.dao.UserDAO"%><%
      String loginstatus = "";
      if (session != null) {
        loginstatus = (String) session.getAttribute("status");
          if (!loginstatus.equals("login")) {
            response.sendRedirect("login.jsp?msg='you are logged out,plz login first'");
        }
      }
%>
<html>
   <%
      String c_user =(String)session.getAttribute("user");
      UserDAO dao =new UserDAO();

   %>
  <head>
  <title>Private Key</title>
   <style type="text/css">
      body{
          margin: 0px;
          background-color: black;
          height: 400px;
      }
      .ow {
            overflow-wrap: break-word;
            word-wrap: break-word;
            hyphens: auto;
      }
      .nav{
          font-size: 30px;
          display: inline;
          float: right;
          margin-right:50px;
      }
      .nav:hover{
          text-decoration: underline;
```

```
        }
        a{
            color: white;
            text-decoration: none;
        }
        .req{
            border-bottom:1px solid gray;
            padding: 10px;
        }
        .breq{
            border-radius:5px;
            float: right;
            background: lime;
        }
    </style>
  </head>
  <body>
    <div style="width: 99.6%;height: 80px; color: white;">
        <p class="nav"><a href="logout" onclick="return
confirm('Continue!')">Logout</a></p>
        <p class="nav"><a href="user_key.jsp"  style="text-decoration:
underline;">Key</a></p>
        <p class="nav"><a href="user_friends.jsp">Friends</a></p>
        <p class="nav"><a href="user_send.jsp">Send Message</a></p>
        <p class="nav"><a href="user_msg.jsp">Messages</a></p>
        <p class="nav"><a href="users_req.jsp">Requests</a></p>
        <p class="nav"><a href="admin_home.jsp">Home</a></p>
    </div>
    <span style="float: right;">Welcome <%=session.getAttribute("user") %></span>
    <h1 style="color: white;margin:0px;margin-top:70px; font-family: cursive;font-size:70px;
text-align: center">Audio SteganoGraphy</h1>
    <h3 style="text-align: center;color: white;">The Art Of Being Invisible</h3>
    <div style="width: 700px; height: 400px; border: 2px solid white; margin-left:
30px;margin-top: 00px; float: left;">
        <img alt="" src="images/home.jpg" height="400" width="700">
    </div>
    <div style="width: 300px;color:blue; height: 200px;background:lightyellow;margin-
right:30px;margin-top:100px; border: 2px solid white;float: right;">
        <h2 style="text-align: center;text-decoration: underline;">Private Key </h2>
        <input type="text" id="tx" style="width:170px;margin-left: 65px;text-align: center;font-
size: 30px;font-weight: bolder;font-family: cursive; background: lightyellow; border: none;"
value="<%=dao.getSkey(c_user) %>" readonly="readonly">
        <br/>
        <button type="button" style="margin-left:100px;margin-top:15px; width: 100px;height:
30px; background: blue;color:white; font-family: cursive;" onclick="myFunction()">copy
key</button>
    </div>


<script>
```

```
function myFunction() {
  var copyText = document.getElementById("tx");
  copyText.select();
  copyText.setSelectionRange(0, 99999)
  document.execCommand("copy");
  alert("Copied the text: " + copyText.value);
}
</script>
  </body>
</html>


<%@page import="java.util.List"%>
<%@page import="com.beans.MessageBean"%>
<%@page import="com.dao.MessageDAO"%>
<%@page import="com.dao.UserDAO"%><%
        String loginstatus = "";
        if (session != null) {
           loginstatus = (String) session.getAttribute("status");
              if (!loginstatus.equals("login")) {
                 response.sendRedirect("login.jsp?msg='you are logged out,plz login first'");
           }
        }

        MessageDAO dao =new MessageDAO();
        UserDAO dao1 =new UserDAO();
        String c_users =(String)session.getAttribute("user");
%>
<html>
  <head>
  <title>Messages</title>
    <style type="text/css">
        body{
            margin: 0px;
            background-color: black;
            height: 400px;
        }
        .ow {
                overflow-wrap: break-word;
                word-wrap: break-word;
                hyphens: auto;
        }
        .nav{
            font-size: 30px;
            display: inline;
            float: right;
            margin-right:50px;
        }
        .nav:hover{
```

```
            text-decoration: underline;
        }
        a{
            color: white;
            text-decoration: none;
        }
        .smsg{
            margin-top:15px;
            margin-left:  20px;
            border-radius:5px;
            background: lime;
        }
        #saudio{
            float: left;
        }
                .loader {
  border: 5px solid #f3f3f3;
  border-radius: 50%;
  border-top: 5px solid #3498db;
  width: 10px;
  height: 10px;
  -webkit-animation: spin 2s linear infinite; /* Safari */
  animation: spin 2s linear infinite;
}

/* Safari */
@-webkit-keyframes spin {
  0% { -webkit-transform: rotate(0deg); }
  100% { -webkit-transform: rotate(360deg); }
}

@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}#l{
    visibility: hidden;
    margin-left: 500px;
}
    </style>
  </head>
  <body>
    <div style="width: 99.6%;height: 80px; color: white;">
        <p class="nav"><a href="logout" onclick="return
confirm('Continue!')">Logout</a></p>
        <p class="nav"><a href="user_key.jsp">Key</a></p>
        <p class="nav"><a href="user_friends.jsp">Friends</a></p>
        <p class="nav"><a href="user_send.jsp">Send Message</a></p>
        <p class="nav"><a href="user_msg.jsp" style="text-decoration:
underline;">Messages</a></p>
```

```
        <p class="nav"><a href="users_req.jsp">Requests</a></p>
        <p class="nav"><a href="admin_home.jsp">Home</a></p>
    </div>
    <div id="l"><p style="margin-top:0px;text-align:center;float:left"
class="loader"></p> <p style="margin-top:20px;display:inline;color:
white;">Decoding Message</p></div>
    <span style="float: right;">Welcome <%=session.getAttribute("user") %></span>
    <h1 style="color: white;margin:0px;margin-top:30px; font-family: cursive;font-size:70px;
    <h3 style="text-align: center;color: white;">The Art Of Being Invisible</h3>

    <div style="background:lightyellow;width: 500px;color:blue; height: 400px;margin-
right:50px; border: 2px solid white;float: right;">
        <h2 style="text-align: center;text-decoration: underline;">Message Received</h2>
        <div style="width: 93%;margin-left:6px;margin-right:6px;padding:10px; height:
77%;border: 1px solid gray;overflow: auto;">
            <%
                List<MessageBean> list1 =dao.getReceivedMsg(c_users);
                if(list1.size()>0){
                int count1=0;
                    for(MessageBean bean:list1){
                        session.setAttribute(count1+"audio1",bean.getMsg());
            %>
                    <div style="padding:10px;;width: 90%; height: 90px; border: 2px solid black;">
                        <div>
                            <p style="display: inline;">Message From :<%=bean.getFrom() %></p>
                            <p style="display: inline;float: right;margin-top: 2px;">Date
:<%=bean.getDate() %></p>
                        </div><br/>
                        <audio id="saudio" controls>
                            <source src ="raudio.jsp?val=<%=count1 %>" type="audio/wav">
                        </audio>
                        <button id="<%=bean.getMsg_id() %>" class="smsg"
onclick="sendInfo('./see?id=<%=bean.getMsg_id() %>')">See Message</button>
                    </div>
            <%
                        count1++;
                    }
                }
            %>
        </div>
    </div>

    <div style="background:lightyellow;width: 500px;color:blue; height: 400px;margin-
right:50px; border: 2px solid white;float: right;">
        <h2 style="text-align: center;text-decoration: underline;">Sent Messages</h2>
        <div style="width: 93%;margin-left:6px;margin-right:6px;padding:10px; height:
77%;border: 1px solid gray;overflow: auto;">
            <%
                List<MessageBean> list =dao.getSentMsg(c_users);
                if(list.size()>0){
```

```
            int count=0;
                for(MessageBean bean:list){
                    session.setAttribute(count+"audio",bean.getMsg());
        %>
                <div style="padding:10px;;width: 90%; height: 90px; border: 2px solid black;">
                    <div>
                        <p style="display: inline;">Message To :<%=bean.getTo() %></p>
                        <p style="display: inline;float: right;margin-top: 2px;">Date
:<%=bean.getDate() %></p>
                    </div><br/>
                    <audio controls>
                        <source src ="audio.jsp?val=<%=count %>" type="audio/wav">
                    </audio>
                </div>
        <%
                    count++;
                }
            }
        %>
    </div>
</div>


<div id="msgbox" style="padding:10px;;border-radius:5px;;margin-top:430px;margin-
left:350px;;background: lightyellow;width: 400px; height: 150px;overflow: auto">
    <h3 style="text-align: center;text-decoration: underline;color: blue;">Message</h3>
    <p class="ow" id="m"></p>
</div>

 <script type="text/javascript">
    var request;
function sendInfo(url) {
    var v =prompt("Enter key");
    document.getElementById("l").setAttribute("style","visibility:visible");
    var key='<%=dao1.getSkey(c_users)%>'
    if(v===key){
    if (window.XMLHttpRequest) {
        request = new XMLHttpRequest();
    } else if (window.ActiveXObject) {
        request = new ActiveXObject("Microsoft.XMLHTTP");
    }
    try {
        request.onreadystatechange = function() {
            if (request.readyState == 4) {
                var res = request.responseText;
                document.getElementById("m").innerHTML="Message : "+res;
                document.getElementById("msgbox").scrollIntoView();
                document.getElementById("l").setAttribute("style","visibility:hidden");
```

```
        };
        request.open("GET", url, true);
        request.send();
    } catch (e) {
        alert("Unable to connect to server");
    }
    }else{
       alert("Invalid key");
    }
}
        </script>
  </body>
</html>

<%@ page language="java" import="java.util.*" pageEncoding="ISO-8859-1"%>
<%@page import="java.io.OutputStream"%>
<%
    String path = request.getContextPath();
    String basePath = request.getScheme() + "://"
            + request.getServerName() + ":" + request.getServerPort()
            + path + "/";
%>

<%
    try {
        int i = Integer.parseInt(request.getParameter("val"));

        byte[] audio = (byte[]) session.getAttribute(i + "audio1");
        OutputStream oImage;
        response.setContentType("audio/wav");
        oImage = response.getOutputStream();
        oImage.write(audio);
        oImage.flush();
        oImage.close();
    } catch (Exception e) {

    }
%>
package com.controller;

import java.io.File;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

```
import com.beans.MessageBean;
import com.dao.MessageDAO;
import com.service.ExtFromAudio;

public class SeeMsg extends HttpServlet {
private static final long serialVersionUID = 1L;


MessageDAO dao =new MessageDAO();
public void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
File dir = new File("D:/project/data");
dir.mkdir();

String mid =request.getParameter("id");
int msg_id=Integer.parseInt(mid);
byte msg[]=null;
int result=0;
String msgdata="";
File finput=null;
File fmsg=null;
try{
MessageBean bean =dao.getMsg(msg_id);
if(bean!=null){
msg =bean.getMsg();
finput =new File(dir+"/"+"temp.wav");
FileOutputStream fos= new FileOutputStream(finput);
fos.write(msg);
fos.flush();
fos.close();

fmsg =new File(dir+"/"+"msg.txt");
result =ExtFromAudio.ext(finput, fmsg);
if(result>0){
FileReader reader =new FileReader(fmsg);
int i =0;
while((i=reader.read())!=-1){
msgdata= msgdata+(char)i;
}
response.getWriter().print(msgdata);
}else{
response.getWriter().print("Internal Error");
}
}
}catch (Exception e) {
response.getWriter().print("Internal Error");
}finally{
finput.delete();
fmsg.delete();
```

```
}

}
}package com.controller;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStream;
import java.util.Date;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.apache.commons.fileupload.FileItem;
import org.apache.commons.fileupload.disk.DiskFileItemFactory;
import org.apache.commons.fileupload.servlet.ServletFileUpload;
import org.apache.commons.io.FilenameUtils;

import com.beans.MessageBean;
import com.dao.MessageDAO;
import com.service.EmbedToAudio;

public class SentFile extends HttpServlet {
private static final long serialVersionUID = 1L;
MessageDAO dao =new MessageDAO();
public void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
File dir = new File("D:/project/data");
dir.mkdir();
String user= (String)request.getSession().getAttribute("user");
String to=null;
String message=null;
int r=0;
byte[] b;
int result=0;
try{
List<FileItem> items = new ServletFileUpload(
new DiskFileItemFactory()).parseRequest(request);
String fileName = FilenameUtils.getName(items.get(1).getName());
to =items.get(0).getString();
InputStream fileContent = items.get(1).getInputStream();
```

```
b = new byte[(int) items.get(1).getSize()];
fileContent.read(b);
message =items.get(2).getString();

File finput =new File(dir+"/"+fileName);
FileOutputStream fos= new FileOutputStream(finput);
fos.write(b);
fos.flush();
fos.close();
File fmsg =new File(dir+"/"+"msg.txt");
FileWriter fw =new FileWriter(fmsg);
fw.write(message);
fw.flush();
fw.close();
File foutput =new File(dir+"/"+"out.wav");
r =EmbedToAudio.embed(finput, fmsg, foutput);

byte[] bout=new byte[(int)foutput.length()];
FileInputStream fis =new FileInputStream(foutput);
fis.read(bout);
fis.close();

MessageBean bean =new MessageBean();
bean.setFrom(user);
bean.setTo(to);
bean.setMsg(bout);
bean.setDate(new java.sql.Date(new Date().getTime()));
result =dao.insertMsg(bean);

finput.delete();
fmsg.delete();
foutput.delete();
}catch (Exception e) {
e.printStackTrace();
}finally{
if(r>0&&result>0)
response.sendRedirect("user_send.jsp?msg='*Message sent'");
else if(r==0)
response.sendRedirect("user_send.jsp?msg='*Audio file to small'");
else
response.sendRedirect("user_send.jsp?msg='*internal  error'");
}
}

}package com.controller;

import java.io.IOException;
```

```
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.dao.UserDAO;

public class AcceptReq extends HttpServlet {
private static final long serialVersionUID = 1L;
UserDAO dao =new UserDAO();
public void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
String frnd =request.getParameter("frnd");
String user =(String)request.getSession().getAttribute("user");
int result =0,result1=0;
try{

result =dao.AcceptReq(user, frnd);
if(result>0)
result1 =dao.cancelReq(frnd, user);

}catch (Exception e) {
e.printStackTrace();
}finally{
if(result1>0){
response.getWriter().print("Operation  Done");
}else{
response.getWriter().print("Operation Failed");
}
}
}
}
```

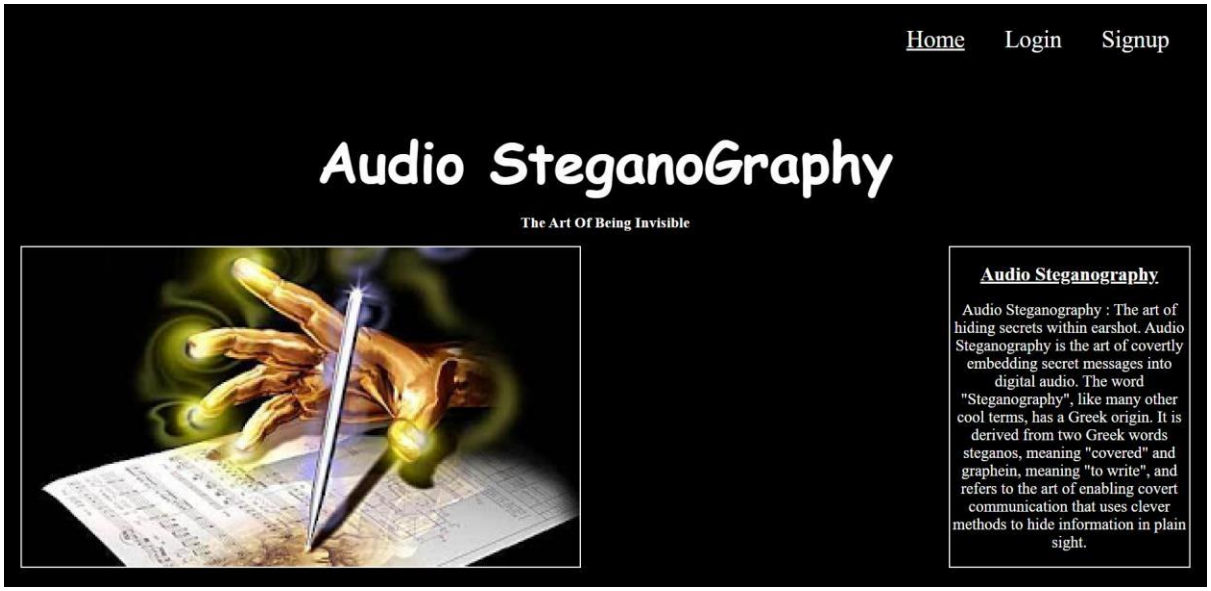
CHAPTER 7

SNAPSHOTS

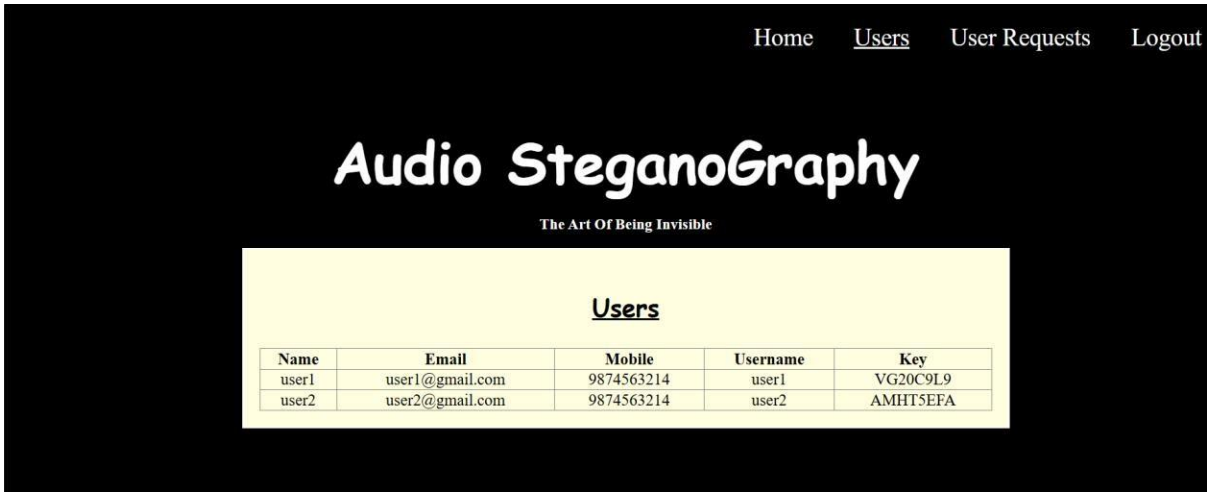
General:

This project is implements like web application using COREJAVA and the Server process is maintained using the SOCKET & SERVERSOCKET and the Design part is played by Cascading Style Sheet

SNAPSHOTS



Home
Users
User Requests
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Users Requests
Name | Email | Mobile | Username | Action
user1 | user1@gmail.com | 9874563214 | user1 | Confirm
user2 | user2@gmail.com | 9874563214 | user2 | Confirm


Home
Users
User Requests
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Users
Name | Email | Mobile | Username | Key
user1 | user1@gmail.com | 9874563214 | user1 | VG20C9L9
user2 | user2@gmail.com | 9874563214 | user2 | AMHT5EFA

Home
Requests
Messages
Send Message
Friends
Key
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Friend Suggestions
user3
Add Friend
Sent Requests
Requests
user1
Accept

Home
Requests
Messages
Send Message
Friends
Key
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Friends
user1
View Profile

Home
Requests
Messages
Send Message
Friends
Key
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Send Message
To Friend
user2
Select Carrier
Choose File
No file chosen
Message
hello user2
Send

Home
Requests
Messages
Send Message
Friends
Key
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Friends
user2
View Profile

Home
Requests
Messages
Send Message
Friends
Key
Logout
*Audio file to small
Audio SteganoGraphy
The Art Of Being Invisible
Send Message
To Friend
user2
Select Carrier
Choose File
file_example_WAV_1MG.wav
Message
Send

Home
Requests
Messages
Send Message
Friends
Key
Logout
Audio SteganoGraphy
The Art Of Being Invisible
Private Key
VG2OC9L9
copy key

CHAPTER 8

SOFTWARE TESTING

8.1 GENERAL

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement

8.2 DEVELOPING METHODOLOGIES

The test process is initiated by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used.The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework from developing the testing methodologies.

8.3 Types of Tests

8.3.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program input produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.3.2 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

8.3.3 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

8.3.4 Performance Test

The Performance test ensures that the output be produced within the time limits,and the time taken by the system for compiling, giving response to the users and request being send to the system for to retrieve the results.

8.3.5 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

8.3.6 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Acceptance testing for Data Synchronization:

- The Acknowledgements will be received by the Sender Node after the Packets are received by the Destination Node
- The Route add operation is done only when there is a Route request in need
- The Status of Nodes information is done automatically in the Cache Updation process

8.2.7 Build the test plan

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identity the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from errors.

CHAPTER 9

APPLICATION

9.1 GENERAL

The above is classified according to the steganographic method. In addition, we can also divide audio Steganography methods into compressed domain and uncompressed domain methods. According to the location of secret information embedded in the audio encoding process, compressed domain audio steganography can be divided into three categories: the pre-embedding, inner-embedding and post-embedding.

9.2 FUTURE ENHANCEMENT

In addition, we also discuss the audio quality evaluation indexes commonly used in literature.

Finally, different steganography methods are evaluated and summarized based on the evaluation indexes. Combined with the characteristics of audio, we believe that the next step should be to consider more air-gapped audio Steganography technology.

CHAPTER 10

CONCLUSION & REFERENCE

10.1 CONCLUSION

This paper reviews the main techniques and evaluation indexes of audio steganography. First, we discuss the differences between cryptography, steganography and watermarking, and

then introduce why audio steganography is important in media steganography. We divide steganography methods into three categories: temporal domain, transform domain and wavelet domain. The technologies in various fields are discussed and their main theories are introduced.

10.2 REFERENCE:

[1] Jayaram P, Ranganatha H R, Anupama H S. Information hiding using audio steganographya survey[J]. The International Journal of Multimedia & Its Applications (IJMA) Vol, 2011, 3: 86-96.

[2] Zamani M, Manaf A A, Ahmad R B, et al. A genetic-algorithmbased approach for audio steganography[J]. World Academy of Science, Engineering and Technology, 2009, 54: 360-3.

[3] Zamani M, Manaf A B A. Genetic algorithm for fragile audio watermarking[J]. Telecommunication Systems, 2015, 59(3): 291-304.

[4] Djebbar F, Ayad B, Hamam H, et al. A view on latest audio Steganography techniques[C]//Innovations in Information Technology (IIT), 2011 International Conference on. IEEE, 2011: 409-414.

[5] Sadek M M, Khalifa A S, Mostafa M G M. Video steganography: a comprehensive review[J]. Multimedia tools and applications, 2015, 74(17): 7063-7094.

[6] Hua G, Huang J, Shi Y Q, et al. Twenty years of digital audio watermarkinga comprehensive review[J]. Signal Processing, 2016, 128: 222-242.

[7] Bazyar M, Sudirman R. A recent review of MP3 based Steganography methods[J]. International Journal of Security and Its Applications, 2014, 8(6): 405-414.

[8] Ni Z, Shi Y Q, Ansari N, et al. Reversible data hiding[J]. IEEE Transactions on circuits and systems for video technology, 2006, 16(3): 354-362.

[9] Balgurgi P P, Jagtap S K. Audio steganography used for secure data transmission[C]//Proceedings of international conference on advances in computing. Springer, New Delhi, 2013: 699-706.

[10] Djebbar F, Ayad B, Meraim K A, et al. Comparative study of digital audio steganography techniques[J]. EURASIP Journal on Audio, Speech, and Music Processing, 2012, 2012(1): 25.

[11] Bender W, Gruhl D, Morimoto N, et al. Techniques for data hiding[J]. IBM systems journal, 1996, 35(3.4): 313-336.

[12] Gruhl D, Lu A, Bender W. Echo hiding[C]//International Workshop on Information Hiding. Springer, Berlin, Heidelberg, 1996: 295-315.

[13] Malviya S, Saxena M, Khare D A. Audio steganography by different methods[J]. International Journal of Emerging Technology and Advanced Engineering Website: www. ijetae. com (ISSN 2250-2459, Volume 2, Issue 7, 2012.

[14] Alwahbani S M H, Elshoush H T I. Hybrid Audio Steganography and Cryptography Method Based on High Least Significant Bit (LSB) Layers and One-Time PadA Novel Approach[C]//Proceedings of SAI Intelligent Systems Conference. Springer, Cham, 2016: 431-453.

[15] Sheikhan M, Asadollahi K, Hemmati E. High quality audio Steganography by floating substitution of lsbs in wavelet domain[J]. World Applied Sciences Journal, 2010, 10(12): 1501-1507.

[16] Totok A. http://cs.nyu.edu/totok/professional/software/tpcw/tpcw.html[Z], 2002.

[17] Deshotels L. Inaudible Sound as a Covert Channel in Mobile Devices[C]//WOOT. 2014.

[18] Wang Q, Ren K, Zhou M, et al. Messages behind the sound: realtime hidden acoustic signal capture with smartphones[C]//Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking. ACM, 2016: 29-41.

[19] Guri M, Solwicz Y, Daidakulov A, et al. Mosquito: Covert ultrasonic transmissions between two air-gapped computers using speaker-tospeaker communication[J]. arXiv preprint arXiv:1803.03422, 2018.

[20] Gruhl D, Lu A, Bender W. Echo hiding[C]//International Workshop on Information Hiding. Springer, Berlin, Heidelberg, 1996: 295-315.

[21] Malviya S, Saxena M, Khare D A. Audio steganography by different methods[J]. International Journal of Emerging Technology and Advanced Engineering Website: www. ijetae. com (ISSN 2250-2459, Volume 2, Issue 7, 2012.

[22] Alwahbani S M H, Elshoush H T I. Hybrid Audio Steganography and Cryptography Method Based on High Least Significant Bit (LSB) Layers and One-Time PadA Novel Approach[C]//Proceedings of SAI Intelligent Systems Conference. Springer, Cham, 2016: 431-453.

[23] Sheikhan M, Asadollahi K, Hemmati E. High quality audio Steganography by floating substitution of lsbs in wavelet domain[J]. World Applied Sciences Journal, 2010, 10(12): 1501-1507.

[24] Totok A. http://cs.nyu.edu/totok/professional/software/tpcw/tpcw.html[Z], 2002.

[25] Deshotels L. Inaudible Sound as a Covert Channel in Mobile Devices[C]//WOOT. 2014.

[26] Wang Q, Ren K, Zhou M, et al. Messages behind the sound: realtime hidden acoustic signal capture with smartphones[C]//Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking. ACM, 2016: 29-41.

[27] Guri M, Solwicz Y, Daidakulov A, et al. Mosquito: Covert ultrasonic transmissions between two air-gapped computers using speaker-tospeaker communication[J]. arXiv preprint arXiv:1803.03422, 2018.